

GeoAgriSense

Technical Documentation

Don't inspect the whole farm. Know where to look first.

Hack for Humanity Harare 2026
Best Use of the Google Gemini API
19 September 2026

Document purpose. This document provides the detailed technical explanation of the GeoAgriSense hackathon prototype, including the problem, architecture, Gemini integration, development process, key learning, testing, limitations, responsible AI considerations and team contributions.

1. Project Title

GeoAgriSense

Tagline: Don't inspect the whole farm. Know where to look first.

GeoAgriSense is an AI-assisted crop-scouting application designed to help farmers record field observations, understand the evidence available, and decide which areas of a farm should be investigated first.

2. Problem Statement

Smallholder horticultural farmers may notice unhealthy-looking plants but still struggle with a practical question: where should I investigate first?

- Photographs may be disconnected from the exact farm location.
- Field notes and observations may not preserve history or crop context.
- Similar observations may occur at different times and places.
- Farmers may have limited time to inspect every area.
- A photograph alone may not show what evidence is missing or which location deserves attention first.

Core problem statement: GeoAgriSense helps a farmer answer: “Where should I investigate first, why should I go there, and what should I check when I arrive?”

The application is designed as crop-scouting decision support, not as a definitive disease-diagnosis system or replacement for an agronomist.

3. Target Users

Primary users

- Smallholder horticultural farmers who need to record crop concerns and decide where to inspect next.
- Farm managers coordinating crop blocks and field inspections.

Secondary users

- Farm scouts who visit locations and record findings.
- Agronomists and extension officers who may review evidence, unresolved concerns and observation history.

4. Proposed Solution

GeoAgriSense combines farm context, geotagged observations, crop photographs, observation history, AI-assisted interpretation, priority logic and scouting information to help determine where the farmer should investigate first.

- Record a concern at a specific location.
- Attach a crop photograph and farmer description.
- Analyse the observation using Gemini Flash 3.8.
- Review visible features, possible explanations, missing information and next checks.
- Assign or review a scouting priority.
- Generate an ordered scouting plan based on available evidence and constraints.
- Record follow-up findings after inspection.

Core journey: Record → Analyse → Review → Prioritise → Scout → Follow Up

5. System Workflow

Farmer Input → Streamlit Interface → Python / Django Application → SQLite + Gemini Flash 3.8 → Processing → Scouting Output → Farmer

1. Farmer records an observation

The farmer enters crop, block/location, symptom description, date/time and optional image/context.

2. Input is validated

The application checks required fields and prepares a valid observation record.

3. Observation is stored

The original farmer report is stored in SQLite before AI analysis.

4. Relevant context is prepared

Python logic prepares crop, description, image and related context for Gemini.

5. Gemini Flash 3.8 analyses the observation

Gemini processes text and image evidence and returns an assessment.

6. Response is processed

The application validates and separates visible features, farmer context, hypotheses, missing information, next checks and priority.

7. Farmer reviews the assessment

AI output is shown as decision support; the farmer remains in control.

8. Scouting priority is determined

Stored records and application logic are used to rank unresolved observations.

9. Farmer asks where to investigate first

The user specifies constraints such as available time or number of stops.

10. Scouting plan is produced

The system returns an ordered, evidence-linked inspection plan.

11. Farmer records a follow-up

New findings become part of the observation history for future decisions.

6. System / Solution Architecture

The prototype uses a Python-based architecture selected for rapid hackathon development and demonstration.

Component	Role
Streamlit	Farmer-facing interface for forms, image uploads, observations, Gemini results and scouting views.
Python	Core application language for data processing, validation, AI integration and decision logic.
Django	Backend/application framework and data-management layer.
SQLite	Prototype relational database for persistent local application records.
Gemini Flash 3.8	Multimodal interpretation and structured AI-assisted scouting support.
Scouting / priority logic	Deterministic application logic used to organise observations and support inspection order.

Architecture flow

Farmer → Streamlit UI → Python/Django logic → SQLite and Gemini Flash 3.8 → validated assessment / scouting logic → farmer-facing output

7. Technologies Used

Technology	Use
Python	Primary language for application logic, data processing, AI integration and validation.
Django	Backend framework and data-management support.
Streamlit	Interactive user interface for the hackathon prototype.
SQLite	Prototype relational data storage.
Google Gemini API	AI platform used by the application.
Gemini Flash 3.8	Model used for multimodal observation analysis and scouting support.

Additional packages should be listed directly from the final requirements.txt in the repository so the technical document and code remain consistent.

8. Gemini API Integration

Gemini is not used as a generic chatbot. It performs a specific technical role in transforming farmer-supplied, multimodal evidence into structured information that can support crop scouting.

8.1 Input sent to Gemini

- Crop type and crop context.
- Farmer symptom description.
- Relevant crop stage or management context when available.
- Crop photograph when supplied.
- Selected related observation context when relevant.

8.2 What Gemini does

- Interprets visible features from the supplied image when image quality permits.
- Separates farmer-reported statements from model observations.
- Provides possible explanations as hypotheses rather than confirmed diagnoses.
- Identifies missing evidence and useful follow-up questions.
- Suggests practical next checks.
- Produces a proposed scouting priority and explanation.
- Can support evidence-linked explanations of where the farmer should investigate first.

8.3 Response returned

- Visible features
- Farmer-reported context
- Possible explanations

- Priority and priority reason
- Evidence quality / uncertainty
- Missing information
- Follow-up questions
- Next checks
- Professional review recommendation
- Limitations

8.4 How the application uses the response

The application parses the Gemini output and presents each part in the appropriate interface section. The original farmer report remains distinct from AI-generated interpretation. AI output can contribute to the proposed scouting priority, while deterministic application logic handles stored records and explicit scoring or workflow rules.

8.5 Why Gemini is useful

The problem is multimodal and context-heavy: farmers may provide natural-language descriptions, crop images and previous observations. Gemini can combine these inputs and turn them into structured, understandable scouting guidance while surfacing uncertainty and missing evidence.

9. Development Process

Problem approach

The team narrowed the problem from broad crop diagnosis to a specific field decision: helping the farmer determine where to investigate first. This created a clearer and safer role for AI.

Team division

- Technical architecture and primary development.
- Product and agriculture research.
- GIS and data support.
- UI/UX design.
- Demo, documentation and testing.

Implementation approach

1. Build the core application and data model.
2. Add observation capture and persistence.
3. Integrate Gemini Flash 3.8 for crop-observation analysis.
4. Add prioritisation and scouting workflow.
5. Refine interface and responsible-AI behaviours.
6. Test the complete demo path and document the final build.

Technical challenges addressed

- Keeping farmer statements separate from AI-generated interpretation.
- Obtaining consistent structured output from a generative model.
- Handling image and text inputs together.

- Persisting observations reliably in a lightweight prototype database.
- Keeping deterministic calculations and records outside the model where possible.
- Designing safeguards so missing evidence becomes an explicit limitation rather than fabricated context.

The team addressed these through structured prompts/output handling, explicit validation, SQLite persistence, clear separation of application logic from model interpretation, and human-review-oriented interface design.

10. Learning

1. A narrower AI problem produced a more useful product

The team initially considered the broader question of crop diagnosis, but the stronger product direction was to focus on a specific field decision: where should the farmer investigate first? Narrowing the problem created a clearer role for Gemini, reduced the risk of overconfident diagnosis, and made the output more actionable for the user.

2. An image becomes more useful when it is connected to context

A crop photograph on its own can be ambiguous. Combining the image with the farmer's description, crop type, location, observation history and management context produces a more useful scouting assessment. This reinforced the importance of treating GeoAgriSense as a context-aware observation system rather than an image-classification tool.

3. Generative AI and deterministic application logic should do different jobs

The team learned that Gemini is most useful for interpreting multimodal evidence, organising uncertainty and explaining recommendations. Stored records, validation, persistence and explicit priority or workflow rules are better handled by application code. Keeping these responsibilities separate makes the system easier to understand, test and audit.

4. Missing information can be a valuable output

A useful AI response does not always need to provide a conclusion. When evidence is weak or incomplete, identifying what is missing and suggesting what the farmer should check next can be more responsible and more useful than forcing a diagnosis. This became an important part of the application's assessment design.

5. Responsible AI is a product-design requirement, not only a disclaimer

Responsible-AI considerations influenced the workflow itself: farmer statements are kept separate from model interpretation, uncertain evidence is surfaced explicitly, unobserved areas are not labelled healthy, and the system avoids autonomous pesticide or fertiliser prescriptions. These safeguards shaped both the technical architecture and the farmer experience.

6. A simple prototype stack helped the team move quickly

Using Python, Django, Streamlit and SQLite allowed the team to move from product definition to a working hackathon prototype quickly while keeping the system understandable. The experience also highlighted which components would need to evolve for production, particularly scalable storage, authentication, offline support and wider field validation.

11. Testing

The final prototype was tested across the primary user journey and the key failure/edge-case scenarios required for the hackathon demonstration. The tests below passed.

Test	Expected behaviour	Result
Application launch	Main interface loads successfully	PASSED
Farm/demo data	Farm and crop information load correctly	PASSED
Observation creation	A farmer observation can be created and saved	PASSED
Image upload	Supported crop image can be uploaded	PASSED
Gemini API integration	Gemini Flash 3.8 request returns a real response	PASSED
AI result rendering	Assessment displays in the application	PASSED
Missing evidence handling	Missing information is surfaced rather than silently fabricated	PASSED
Priority output	Observation receives a proposed scouting priority	PASSED
SQLite persistence	Saved records remain available in the prototype database	PASSED
Scouting workflow	System produces an ordered inspection/scouting output	PASSED
Follow-up workflow	Follow-up findings can be recorded	PASSED
Text-only / limited evidence handling	The system can handle observations without relying on invented visual findings	PASSED
API/error handling	Saved observation is preserved if AI processing fails	PASSED

Testing outcome: The core hackathon demo path is functional and ready for presentation.

12. Limitations

- Gemini outputs may still be incorrect or incomplete and must not be treated as confirmed agronomic diagnoses.
- Image quality can materially affect what the model can observe.
- The priority approach is a prototype decision-support mechanism, not a validated agronomic risk model.

- SQLite is suitable for the hackathon prototype but not the final choice for a large concurrent multi-user service.
- Gemini-powered analysis depends on network/API availability.
- The prototype has not yet undergone broad field validation across farms, crops, regions and image conditions.
- No claim is made that the prototype improves yield, prevents disease or reduces crop losses without further evaluation.

13. Responsible AI / Risk Considerations

Privacy

Farm location, images and operational records can be sensitive. A production system would require authentication, access controls, consent, secure storage and retention rules.

Inaccurate AI outputs

AI-generated interpretations may be wrong. The interface separates visible evidence, farmer statements, hypotheses and missing information.

Bias and generalisation

Performance may vary by crop, region, image conditions, language and farming system. Results should not be assumed to generalise universally.

Misuse / unsafe treatment advice

The system focuses on scouting and evidence gathering rather than autonomously prescribing pesticides, fertilisers or application rates.

Human oversight

Farmers and professionals remain responsible for decisions. AI recommendations are decision support, not autonomous instructions.

False confidence from maps

No observations does not mean an area is healthy. Unobserved areas remain unassessed.

Spatial inference

Nearby similar observations do not prove disease spread or common causation.

Prompt/input safety

Farmer notes and uploaded content are treated as evidence, not as trusted instructions to the model.

14. Future Improvements

- Offline-first observation capture and delayed synchronisation for farms with unreliable connectivity.
- More advanced farm mapping and route visualisation.
- Weather data as supporting context.

- Satellite or remote-sensing layers where appropriate and validated.
- Multi-user accounts for farmers, scouts and agronomists.
- Verified agronomist review workflows.
- Local-language interfaces after appropriate translation and usability testing.
- More robust AI evaluation against agronomist-labelled examples.
- Migration from SQLite to a production-grade database for scalable deployment.
- Field pilots with farmers and agricultural professionals to validate usefulness and workflow fit.

15. Team Members & Contributions

Butho Dube

Role: Technical Architect / Primary Developer

- Overall technical architecture
- Primary prototype development
- Backend and application logic
- Gemini integration
- Technical integration and coordination
- Implementation of the main application workflow

Rufaro Nyakudya

Role: Product & Agriculture / Demo & Documentation

- Farmer problem definition and product research
- Agricultural use cases and realistic farmer scenarios
- Recommendation logic and responsible-AI considerations
- Technical and product documentation
- Demo preparation and presentation support
- Testing support and build tracking

Thembelihle Ndebele

Role: Product & Agriculture / Demo & Documentation

- Agricultural and product research
- Farmer use-case development
- Farmer workflow refinement
- Responsible-AI considerations
- Documentation and demo preparation
- Testing and review

Shaun Jeranyama

Role: GIS / Data Assistant

- GIS and spatial-data support
- Farm mapping support

- Crop-block and farm-location data
- Observation-location data
- Geographic data preparation
- Support for mapping/location-based functionality

Tichaona B Wutete

Role: UI/UX

- User-interface and user-experience design
- Visual consistency and application layout
- Farmer-facing user flows
- Usability of observation and scouting workflows
- Clear presentation of AI assessments and scouting information
- Prototype visual design

Shared team contribution

The team collaborated across research, development, GIS, agriculture, user experience, testing, documentation and demo preparation to deliver the hackathon prototype.

Final Technical Stack

Layer	Technology
Programming language	Python
Backend / application framework	Django
User interface	Streamlit
Database	SQLite
AI model	Gemini Flash 3.8
AI platform	Google Gemini API

Final System Summary

GeoAgriSense follows the flow: Farmer observation → Streamlit interface → Python/Django processing → SQLite storage → Gemini Flash 3.8 analysis → application logic → explainable scouting recommendation → farmer inspection → follow-up observation.

The solution is intentionally designed as decision support rather than an autonomous agricultural decision-maker. Its central question remains: Where should I investigate first?